# Lab Sheet 01

## Part 01: Fundamentals

### 1. Printing and Comments

Let's start by printing some values and adding comments to explain the code.

```
# Printing a number (Here the comment is shown with #)

print(200)


# Printing a string

print("Dog")
```

---

### 2. Mathematical Operations

Perform basic arithmetic operations such as addition, multiplication, division, exponentiation, remainder, and integer division.

```
# Arithmetic operations

print(1 + 2)  # Addition

print(2 * 3)  # Multiplication

print(4 / 2)  # Division

print(4 ^ 2)  # Exponentiation using ^

print(4 ** 2) # Exponentiation using **

print(5 %% 2) # Remainder

print(5 %/% 2) # Integer division
```

---

### 3. Relational Operations

Compare values using relational operators like greater than, less than, equal to, and not equal to.

```
# Relational operations

print(20 > 10)   # Greater than

print(30 < 20)   # Less than

print(50 >= 30)  # Greater than or equal to

print(10 == 10)  # Equal

print(10 != 10)  # Not equal
```

---

### 4. Logical Operations

Combine conditions using logical operators such as AND, OR, and NOT.

```
# Logical operations

print((20 > 10) && (30 <= 50))  # AND

print((20 < 10) || (20 == 20))  # OR

print(!(10 == 10))            # NOT
```

---

### 5. Variables

Define and manipulate variables, and check which variables exist in the environment.

```
# Assign values

x = 10

y = 20
```

```
# Addition
z = x + y
print(z)

# List variables
print(ls())

# Remove variable x
rm(x)
print(ls())

# Remove all variables
rm(list = ls())
print(ls())
```

---

**6. Mathematical Functions**

Explore built-in mathematical functions like summation, absolute value, square root, logarithms, exponentials, and rounding.

```
# Mathematical functions
print(sum(10, 20))
print(abs(-20))
print(sqrt(25))
print(log(10))  # Natural log
```

```
print(exp(5))   # Exponential
print(round(3.245))
print(round(10.35, 1))
```

---

**7. Primitive Data Types**

Explore different data types in R, such as numeric, integer, character, logical, and complex. Verify their types using the class() function.

```
# Data types
num = 1.23
int = 12L
char = "Cat"
log = TRUE
comp = 23 + 3i

# Check types
print(class(num))
print(class(int))
print(class(char))
print(class(log))
print(class(comp))
```

---

**8. Casting Data Types**

Convert between data types, such as numeric to character, character to numeric, and logical to numeric.

```
# Casting data types
x = 12
y = as.character(x)
print(y)
print(class(y))

x = "12"
y = as.numeric(x)
print(y)
print(class(y))

x = TRUE
y = as.numeric(x)
print(y)
print(class(y))
```

---

**9. User Input**

Accept user input and process it, such as converting age input from character to numeric.

```
# User input
name = readline(prompt = "Enter your name: ")
age = readline(prompt = "Enter your age: ")
```

```
print(name)
print(age)

# Convert age to numeric
age = as.numeric(age)
print(age)
```

---

**Part 02: Compound Data Structures**

---

**1. Vectors**

**Creating Vectors**

1. Create numeric, character, and mixed vectors using c(). Observe coercion when mixing types.
2. Generate sequences using 1:10, seq(), and rep().

```
# Examples of vector creation
numeric_vector = c(12, 22, 23, 34, 35)
character_vector = c("Dog", "Cat", "Rat")
mixed_vector = c(12, "Man", 23)
sequence_vector = seq(10, 50, by = 5)
repeated_vector = rep("Dog", 5)
```

**Indexing and Slicing**

Access specific elements, slices, and apply Boolean masking.

```
# Indexing and slicing
```

```
vector = c(23, 33, 34, 32, 45, 50, 65)

element = vector[1]

slice = vector[1:3]

boolean_masked = vector[vector > 40]
```

**Element-Wise Operations**

Perform arithmetic and comparisons on vectors.

```
# Element-wise operations

vector1 = c(2, 3, 4)

vector2 = c(3, 4, 5)

sum_vector = vector1 + vector2

comparison = vector1 > 2
```

**Summary Functions**

Explore functions like sum(), mean(), sd(), and range().

```
# Summary functions

vector = c(2, 3, 4, 5)

print(sum(vector))

print(mean(vector))

print(sd(vector))

print(range(vector))
```

**Other Vector Functions**

Use append(), sort(), unique(), and more for practical operations.

```
# Additional vector functions

vector = c(20, 10, 20, 30)
```

```
appended_vector = append(vector, 40)

sorted_vector = sort(vector)

unique_values = unique(vector)

sampled_values = sample(vector, 2)
```

---

**2. Matrices**

**Creating Matrices**

1. Create matrices using matrix(). Explore row-wise and column-wise filling.

```
# Matrix creation

matrix1 = matrix(1:6, nrow = 3, ncol = 2)

matrix2 = matrix(1:6, nrow = 3, ncol = 2, byrow = TRUE)
```

**Matrix Properties**

Determine dimensions, structure, and summaries.

```
# Matrix properties

print(dim(matrix1))

print(summary(matrix1))
```

**Merging Matrices**

Combine matrices using cbind() and rbind().

```
# Merging matrices

matrix1 = matrix(1:4, nrow = 2)

matrix2 = matrix(5:8, nrow = 2)

horizontal_merge = cbind(matrix1, matrix2)
```

```
vertical_merge = rbind(matrix1, matrix2)
```

**Matrix Indexing and Operations**

Access rows, columns, and perform matrix arithmetic.

```
# Matrix indexing
matrix1 = matrix(1:9, nrow = 3)
print(matrix1[1, ])  # First row
print(matrix1[, 2])  # Second column
print(matrix1[1:2, 2:3])


# Matrix operations
matrix1 = matrix(c(1, 2, 3, 4), nrow = 2)
matrix2 = matrix(c(5, 6, 7, 8), nrow = 2)
matrix_addition = matrix1 + matrix2
matrix_subtraction = matrix1 - matrix2
matrix_multiplication = matrix1 * matrix2  # Element-wise
matrix_dot_product = matrix1 %*% matrix2  # Matrix multiplication
matrix_transpose = t(matrix1)
print(matrix_addition)
print(matrix_transpose)
```

**Advanced Matrix Operations**

Calculate determinants, inverse, and diagonal elements.

```
# Determinants and inverse
matrix3 = matrix(c(4, 7, 2, 6), nrow = 2)
```

```
det_value = det(matrix3)
inverse_matrix = solve(matrix3)
print(det_value)
print(inverse_matrix)

# Diagonal elements
diag_elements = diag(matrix3)
print(diag_elements)
```

---

**3. Factors**

**Unordered Factors**

Convert categorical data into factors.

```
# Unordered factors
gender = factor(c("Male", "Female", "Male"))
print(gender)
```

**Ordered Factors**

Specify an order for factor levels.

```
# Ordered factors
levels = factor(c("Low", "Medium", "High"), levels = c("Low", "Medium", "High"), ordered = TRUE)
print(levels)
```

---

### 4. Lists

#### Creating Lists

Combine multiple data types into a list.

```
# List creation
my_list = list(
  vector = c(1, 2, 3),
  matrix = matrix(1:4, nrow = 2),
  char = "Hello"
)
print(my_list)
```

---

### 5. Data Frames

#### Manual Creation

Create data frames from vectors.

```
# Data frame creation
name = c("Kane", "Jane", "David")
age = c(23, 33, 34)
marks = c(89, 78, 88)
df = data.frame(name, age, marks)
print(df)
```

#### Accessing Columns

Access columns using [] and $.

```
# Accessing columns
```

```
print(df["name"])

print(df$name)
```

**Indexing and Slicing**

Access specific rows and columns.

```
# Indexing and slicing

print(df[1, 1])

print(df[1:2, ])
```

**Modifying Data Frames**

Change elements, add, and remove columns.

```
# Modifying data frames

df$grade = ifelse(df$marks > 80, "A", "B")

print(df)
```

**Boolean Masking**

Filter rows based on conditions.

```
# Boolean masking

filtered_df = df[df$marks > 80, ]

print(filtered_df)
```

**Data Frame Functions**

Explore head(), tail(), dim(), nrow(), ncol(), colnames(), rownames(), rowSums(), colSums(), rowMeans(), colMeans(), summary(), and str().

```
# Data frame functions

print(head(df))

print(tail(df))
```

```
print(dim(df))
print(nrow(df))
print(ncol(df))
print(colnames(df))
print(rownames(df))
print(rowSums(df[, c("age", "marks")]))
print(colSums(df[, c("age", "marks")]))
print(rowMeans(df[, c("age", "marks")]))
print(colMeans(df[, c("age", "marks")]))
print(summary(df))
print(str(df))
```

**Changing Columns to Factors**

Convert categorical columns to factors in a data frame.

```
# Changing to factors
df$name = factor(df$name)
print(str(df))
```

**Working with CSV Files**

Import and explore data from CSV files.

```
# Reading CSV files
data = read.csv("data.csv")
print(head(data))
```

**Additional Data Frame Operations**

Perform more column operations such as creating new columns based on conditions, renaming columns, and applying functions across rows or columns.

```
# Adding new columns

data$new_column = data$marks * 2

print(data)


# Renaming columns

colnames(data)[colnames(data) == "marks"] = "Scores"

print(colnames(data))


# Applying a function

data$scaled_marks = scale(data$Scores)

print(data)
```

**Using the Apply Family of Functions**

Apply functions to rows or columns using apply(), lapply(), and sapply().

```
# Apply functions

data$average_score = apply(data[, c("age", "Scores")], 1, mean)

print(data)


column_sums = sapply(data[, c("age", "Scores")], sum)

print(column_sums)
```

```
list_structure = lapply(data[, c("age", "Scores")], summary)

print(list_structure)
```

**Part 03: Flow Control Structures**

**1. If Statements**

**01)** Write a program to check if a number is positive. Print "Positive" if it is greater than zero.

```
number = 5

if(number > 0) {

  print("Positive")

}
```

**02)** Write a program to check if a given year is a leap year. A year is a leap year if it is divisible by 4 and not divisible by 100, except when it is divisible by 400.

```
year = 2024

if((year %% 4 == 0 && year %% 100 != 0) || (year %% 400 == 0)) {

  print("Leap Year")

}
```

**03)** Write a program to determine if a number is within the range of 10 to 50 (inclusive).

```
number = 25

if(number >= 10 && number <= 50) {

  print("Number is in the range")

}
```

---

**2. If-Else Statements**

**01)** Write a program to check if a number is even or odd.

```
number = 8
if(number %% 2 == 0) {
  print("Even")
} else {
  print("Odd")
}
```

**02)** Write a program to compare two numbers and print the larger one.

```
a = 15
b = 20
if(a > b) {
  print("a is larger")
} else {
  print("b is larger")
}
```

**03)** Write a program to check if a user-entered username and password are correct. Print "Access Granted" if both match; otherwise, print "Access Denied".

```
username = "user"
password = "pass123"

input_user = readline(prompt = "Enter username: ")
input_pass = readline(prompt = "Enter password: ")
```

```
if(input_user == username && input_pass == password) {
  print("Access Granted")
} else {
  print("Access Denied")
}
```

---

### 3. If-Elseif-Else Statements

**01)** Write a program to assign grades based on marks:

- Marks >= 80: "A"
- Marks >= 65: "B"
- Marks >= 50: "C"
- Otherwise: "F"

```
marks = 72
if(marks >= 80) {
  grade = "A"
} else if(marks >= 65) {
  grade = "B"
} else if(marks >= 50) {
  grade = "C"
} else {
  grade = "F"
}
print(grade)
```

**02)** Write a program to categorize a number as "Positive", "Negative", or "Zero".

```
number = -7
if(number > 0) {
  print("Positive")
} else if(number < 0) {
  print("Negative")
} else {
  print("Zero")
}
```

**03)** Write a program to calculate the discount percentage based on purchase amount:

- = 500: 20%
- = 200: 10%
- Otherwise: 5%

```
amount = 350
if(amount >= 500) {
  discount = 20
} else if(amount >= 200) {
  discount = 10
} else {
  discount = 5
}
print(paste("Discount:", discount, "%"))
```

---

**4. For Loops**

**01)** Print all numbers from 1 to 10.

```
for(i in 1:10) {
  print(i)
}
```

**02)** Calculate and print the factorial of a number.

```
number = 5
factorial = 1
for(i in 1:number) {
  factorial = factorial * i
}
print(factorial)
```

**03)** Print the squares of all elements in a given vector.

```
numbers = c(2, 4, 6, 8, 10)
for(n in numbers) {
  print(n^2)
}
```

---

**5. While Loops**

**01)** Write a program to print the first 10 multiples of 3 using a while loop.

```
number = 1
```

```
while(number <= 10) {
  print(number * 3)
  number = number + 1
}
```

**02)** Keep prompting the user to enter a number until they provide a positive number.

```
number = -1
while(number <= 0) {
  number = as.numeric(readline(prompt = "Enter a positive number: "))
}
print("Thank you!")
```

**03)** Simulate a simple ATM PIN validation system. The user must enter the correct PIN to exit the loop.

```
pin = 1234
entered_pin = 0
while(entered_pin != pin) {
  entered_pin = as.numeric(readline(prompt = "Enter your PIN: "))
  if(entered_pin != pin) {
    print("Incorrect PIN. Try again.")
  }
}
print("Access Granted!")
```

# Lab Sheet 02

## Part 01 : Fundamentals in R

### 1. Functions in R

### What are Functions?

Functions in R are reusable blocks of code designed to perform specific tasks. They can take inputs (arguments), perform operations, and return results.

---

### 2. Tasks for User-Defined Functions

1. Write a function that prints "Hello, R!" to the console.

```
# Function definition
hello_function = function() {
  print("Hello, R!")
}


# Function call
hello_function()
```

2. Create a function that takes two numbers as input and returns their product.

```
# Function definition
multiply_numbers = function(a, b) {
  return(a * b)
}
# Function call
result = multiply_numbers(4, 5)
```

```
print(result)
```

3. Write a function that calculates the square of a number and adds 10 to the result.

```
# Function definition
calculate_square = function(x) {
  return((x^2) + 10)
}

# Function call
square_result = calculate_square(6)
print(square_result)
```

---

**3. Tasks for Anonymous Functions**

4. Create an anonymous function to calculate the cube of a number and call it immediately.

```
# Anonymous function
print((function(x) x^3)(3))
```

5. Use an anonymous function with sapply() to double the values in a numeric vector.

```
# Numeric vector
values = c(5, 10, 15, 20)

# Using sapply with an anonymous function
result = sapply(values, function(x) x * 2)
```

```
print(result)
```

6. Apply an anonymous function to calculate the sum of squares for each row in a data frame.

```
# Data frame
data_frame = data.frame(A = c(2, 4, 6), B = c(3, 5, 7))


# Using apply with an anonymous function
row_sums = apply(data_frame, 1, function(x) sum(x^2))
print(row_sums)
```

---

**4. Advanced Task with Functions**

7. Create a function that computes the perimeter of a rectangle by calling another function to calculate its area. The perimeter function should take the length and width as inputs.

```
# Area function
calculate_area = function(length, width) {
  return(length * width)
}


# Perimeter function
calculate_perimeter = function(length, width) {
  area = calculate_area(length, width)
  return(2 * (length + width))
}
```

```
# Function call
perimeter_result = calculate_perimeter(5, 3)
print(perimeter_result)
```

---

**5. Libraries in R**

**What are Libraries?**

Libraries in R are collections of pre-written functions and datasets that extend the functionality of R. Popular libraries include dplyr, ggplot2, and shiny.

**Examples of Libraries**

1. **Data Manipulation**
    - Library: dplyr
2. **Visualization**
    - Library: ggplot2
3. **Machine Learning**
    - Library: randomForest
4. **Web Applications**
    - Library: shiny

**Additional Notes**

- To install a library: install.packages("library_name")
- To check available datasets: data()
- Example: Load the built-in iris dataset and view its structure.
- data("iris")

- str(iris)

**Part 02 : Statistical Summaries with R**

**1. Descriptive Statistical Summaries**

Descriptive statistics help summarize and describe the main features of a dataset. The iris dataset contains information about the sepal length, sepal width, petal length, and petal width of different species of flowers.

**Basic Summaries**

```
# Load the iris dataset

data("iris")

head(iris)  # View the first few rows


# Summary statistics

summary(iris)


# Mean of each numerical column

sapply(iris[, 1:4], mean)


# Standard deviation of each numerical column

sapply(iris[, 1:4], sd)


# Range of each numerical column

sapply(iris[, 1:4], range)
```

---

## 2. Visualizations Using Base R

### 2.1 Bar Charts

Create a bar chart to display the count of each species in the dataset.

```
# Bar chart

species_count = table(iris$Species)

barplot(species_count, main = "Count of Species", col = "lightblue")
```

### 2.2 Pie Charts

Visualize the proportion of each species using a pie chart.

```
# Pie chart

pie(species_count, main = "Proportion of Species", col = rainbow(3))
```

### 2.3 Histograms

Create a histogram for Sepal Length to understand its distribution.

```
# Histogram

hist(iris$Sepal.Length, main = "Histogram of Sepal Length", col = "lightgreen",
xlab = "Sepal Length")
```

### 2.4 Box Plots

Visualize the distribution of Sepal Length using a box plot.

```
# Box plot

boxplot(iris$Sepal.Length, main = "Box Plot of Sepal Length", col = "pink")
```

### 2.5 Multiple Bar Charts

Create a grouped bar chart to compare Sepal Length across species.

```
# Multiple bar charts
sepal_mean = tapply(iris$Sepal.Length, iris$Species, mean)
barplot(sepal_mean, main = "Mean Sepal Length by Species", col = c("red", "blue", "green"), beside = TRUE)
```

**2.6 Side-by-Side Box Plots**

Create side-by-side box plots for Sepal Length across species.

```
# Side-by-side box plots
boxplot(Sepal.Length ~ Species, data = iris, main = "Sepal Length by Species", col = c("red", "blue", "green"))
```

**2.7 Scatter Plots**

Visualize the relationship between Sepal Length and Petal Length using a scatter plot.

```
# Scatter plot
plot(iris$Sepal.Length, iris$Petal.Length, main = "Scatter Plot: Sepal vs Petal Length",
    xlab = "Sepal Length", ylab = "Petal Length", col = iris$Species, pch = 19)
```

---

**3. Correlation**

**What is Correlation?**

Correlation measures the strength and direction of a linear relationship between two variables. The correlation coefficient ranges from -1 to 1:

- **1**: Perfect positive correlation
- **-1**: Perfect negative correlation
- **0**: No correlation

**Calculating Correlation in R**

```
# Correlation between Sepal Length and Petal Length
correlation = cor(iris$Sepal.Length, iris$Petal.Length)
print(correlation)
```

---

**4. Correlation Matrix**

**What is a Correlation Matrix?**

A correlation matrix displays the correlation coefficients between multiple variables in a dataset.

**Calculating Correlation Matrix**

```
# Correlation matrix for numerical columns
cor_matrix = cor(iris[, 1:4])
print(cor_matrix)
```

**Visualizing Correlation Matrix**

```
# Visualize the correlation matrix
image(1:4, 1:4, cor_matrix, axes = FALSE, main = "Correlation Matrix", col = heat.colors(10))
axis(1, at = 1:4, labels = colnames(iris)[1:4])
axis(2, at = 1:4, labels = colnames(iris)[1:4])
```

# Lab Sheet 03

## Part 01 : Probability Distributions

## Sample Codes for Binomial Distribution in R

### 1. dbinom() - Probability Mass Function (PMF)

Calculates the probability of exactly x successes in n trials with success probability p.

dbinom(x, size, prob)

- **x** – Number of successes
- **size** – Number of trials
- **prob** – Probability of success in each trial

**Example:**

Probability of exactly 3 successes in 10 trials with p = 0.5

```
dbinom(3, size = 10, prob = 0.5)
```

### 2. pbinom() - Cumulative Distribution Function (CDF)

Calculates the cumulative probability up to x successes.

pbinom(q, size, prob, lower.tail = TRUE)

- **q** – Number of successes
- **size** – Number of trials
- **prob** – Probability of success
- **lower.tail** – TRUE for $P(X \leq x)$, FALSE for $P(X > x)$

**Example:**

Probability of 3 or fewer successes in 10 trials with p = 0.5

```
pbinom(3, size = 10, prob = 0.5)
```

Probability of more than 3 successes

pbinom(3, size = 10, prob = 0.5, lower.tail = FALSE)

### 3. qbinom() - Quantile Function (Inverse CDF)

Finds the number of successes corresponding to a given cumulative probability.

qbinom(p, size, prob, lower.tail = TRUE)

- **p** – Probability value (between 0 and 1)
- **size** – Number of trials
- **prob** – Probability of success
- **lower.tail** – TRUE for lower quantile, FALSE for upper quantile

**Example:**

Number of successes corresponding to 0.5 cumulative probability

qbinom(0.5, size = 10, prob = 0.5)

### 4. rbinom() - Random Sampling

Generates random samples from a binomial distribution.

rbinom(n, size, prob)

- **n** – Number of random samples
- **size** – Number of trials
- **prob** – Probability of success

**Example:**

Generate 10 random values from a binomial distribution

rbinom(10, size = 10, prob = 0.5)

## 1. Binomial Distribution

**Question 01)** A factory produces light bulbs, and the probability of a bulb being defective is 0.1. Out of a batch of 10 bulbs, what is the probability that

1. Exactly 2 bulbs are defective
2. At most 2 bulbs are defective

```
n = 10

p = 0.1

prob_exactly_2 = dbinom(2, size = n, prob = p)

prob_at_most_2 = pbinom(2, size = n, prob = p)


print(prob_exactly_2)

print(prob_at_most_2)
```

**Question 02)** A marketing survey finds that 40% of customers prefer brand A. In a random sample of 15 customers, what is the probability that

1. Exactly 6 customers prefer brand A
2. More than 6 customers prefer brand A

---

## 2. Hypergeometric Distribution

**Question 01)** A deck of cards has 52 cards, including 13 hearts. If 5 cards are drawn without replacement, what is the probability of getting exactly 2 hearts

```
N = 52

K = 13

n = 5
```

```
prob_2_hearts = dhyper(2, K, N - K, n)

print(prob_2_hearts)
```

**Question 02)** In a batch of 20 items, 7 are defective. If 6 items are randomly chosen, what is the probability that

1. Exactly 3 items are defective
2. At least 3 items are defective

---

### 3. Poisson Distribution

**Question 01)** A call center receives an average of 5 calls per minute. What is the probability of receiving exactly 7 calls in a minute

```
lambda = 5

prob_7_calls = dpois(7, lambda)

print(prob_7_calls)
```

**Question 02)** If a website gets an average of 10 hits per second, what is the probability that

1. No hits are received in a second
2. At least 5 hits are received in a second

---

### 4. Geometric Distribution

**Question 01)** A die is rolled until a 6 appears. What is the probability that it takes exactly 4 rolls to get a 6

```
p = 1/6

prob_4_rolls = dgeom(3, prob = p)

print(prob_4_rolls)
```

**Question 02)** The probability of a customer making a purchase at a store is 0.2. What is the probability that the first purchase occurs on the 5th customer

---

### 5. Negative Binomial Distribution

**Question 01)** A basketball player has a 60% chance of making a free throw. What is the probability that the player makes the 5th successful shot on the 8th attempt

```
r = 5

p = 0.6

prob_5th_success = dnbinom(3, size = r, prob = p)

print(prob_5th_success)
```

**Question 02)** A factory produces widgets, and 10% of them are defective. What is the probability that the 3rd defective widget is found on the 7th inspection

---

### 6. Exponential Distribution

**Question 01)** The average time between arrivals of buses at a station is 10 minutes. What is the probability that the next bus arrives within 5 minutes

```
rate = 1/10

prob_within_5 = pexp(5, rate = rate)

print(prob_within_5)
```

**Question 02)** A machine part has an average lifespan of 100 hours. What is the probability that it fails after 120 hours

---

### 7. Normal Distribution

**Question 01)** The heights of students in a class are normally distributed with a mean of 170 cm and a standard deviation of 10 cm. What is the probability that a randomly selected student is taller than 180 cm

```
mean = 170

sd = 10

prob_taller_180 = pnorm(180, mean, sd, lower.tail = FALSE)

print(prob_taller_180)
```

**Question 02)** IQ scores are normally distributed with a mean of 100 and a standard deviation of 15. What is the probability that a person's IQ is between 85 and 115

**Question 03)** A company's weekly sales are normally distributed with a mean of $50,000 and a standard deviation of $5,000. What is the probability that the sales are less than $40,000

**Question 04)** What is the z-score for a value of $60,000 in the above case

**Question 05)** Given the above distribution, find the 95th percentile of weekly sales

---

**8. Gamma Distribution**

**Question 01)** The time to complete a task follows a Gamma distribution with a shape parameter of 2 and a scale parameter of 3. What is the probability of completing the task in less than 5 units of time

```
shape = 2

scale = 3

prob_less_5 = pgamma(5, shape = shape, scale = scale)

print(prob_less_5)
```

**Question 02)** If the same task follows the above Gamma distribution, what is the density value at 4 units of time

---

### 9. Beta Distribution

**Question 01)** The proportion of defective items in a batch follows a Beta distribution with shape parameters α = 2 and β = 5. What is the probability that the proportion is less than 0.3

```
alpha = 2

beta = 5

prob_less_0_3 = pbeta(0.3, shape1 = alpha, shape2 = beta)

print(prob_less_0_3)
```

**Question 02)** What is the mode of the Beta distribution in Question 1

### Part 02 : Data Cleaning in R

Data cleaning is an essential step in data analysis, ensuring your dataset is accurate, consistent, and suitable for analysis. This lab sheet covers three key aspects of data cleaning in R

- Handling Missing Values
- Handling Duplicates
- Handling Outliers

We will use manually created data frames to demonstrate these concepts.

### 1. Handling Missing Values

**Sample Dataset with Missing Values**

```
df = data.frame(
  ID = 1:8,
  Name = c("Alice", "Bob", "Charlie", "David", NA, "Frank", "George", "Helen"),
  Age = c(25, NA, 30, 45, 50, NA, 34, 29),
  Score = c(85, 90, NA, 88, 75, 80, 95, NA)
)
print(df)
```

**Detecting Missing Values**

```
# Check for NA values
table(is.na(df))

# Column-wise count of missing values
colSums(is.na(df))
```

**Handling Missing Values**

**Method 1:** Removing Rows with Missing Values

```
df_clean = na.omit(df)
print(df_clean)
```

**Method 2:** Filling Missing Values with Mean/Median/Mode

```
# Fill Age column with mean
df$Age[is.na(df$Age)] = mean(df$Age, na.rm = TRUE)

# Fill Score column with median
df$Score[is.na(df$Score)] = median(df$Score, na.rm = TRUE)
```

```
print(df)
```

**Method 3:** Using mice Library for Advanced Imputation

```
library(mice)
df_imputed = mice(df, method = "pmm", m = 1)
df_cleaned = complete(df_imputed)
print(df_cleaned)
```

**2. Handling Duplicates**

**Sample Dataset with Duplicates**

```
df_duplicates = data.frame(
  ID = c(1, 2, 3, 3, 4, 5, 5, 6),
  Name = c("Alice", "Bob", "Charlie", "Charlie", "David", "Frank", "Frank",
"George"),
  Age = c(25, 30, 30, 30, 40, 50, 50, 60)
)
print(df_duplicates)
```

**Detecting Duplicates**

```
# Identify duplicates
duplicated(df_duplicates)

# Display duplicate rows
df_duplicates[duplicated(df_duplicates), ]
```

**Removing Duplicates**

**Method 1:** Remove All Duplicate Rows

```
df_unique = df_duplicates[!duplicated(df_duplicates), ]
print(df_unique)
```

**Method 2:** Keep Only the Last Occurrence

```
df_last = df_duplicates[!duplicated(df_duplicates, fromLast = TRUE), ]
print(df_last)
```

**3. Handling Outliers**

**Sample Dataset with Outliers**

```
df_outliers = data.frame(
  Student = c("A", "B", "C", "D", "E", "F", "G"),
  Scores = c(78, 85, 90, 95, 150, 200, 88)  # 150 and 200 are outliers
)
print(df_outliers)
```

**Detecting Outliers**

**Method 1:** Using Boxplot

```
boxplot(df_outliers$Scores, main = "Boxplot for Scores")
```

**Method 2:** Using IQR (Interquartile Range)

```
Q1 = quantile(df_outliers$Scores, 0.25)
Q3 = quantile(df_outliers$Scores, 0.75)
IQR_value = Q3 - Q1
```

```
# Identifying Outliers

outlier_condition = df_outliers$Scores < (Q1 - 1.5 * IQR_value) |
df_outliers$Scores > (Q3 + 1.5 * IQR_value)

print(df_outliers[outlier_condition, ])
```

**Handling Outliers**

**Method 1:** Removing Outliers

```
df_no_outliers = df_outliers[!outlier_condition, ]

print(df_no_outliers)
```

**Method 2:** Capping Outliers (Winsorizing)

```
# Cap extreme values at upper and lower thresholds

df_outliers$Scores[df_outliers$Scores > (Q3 + 1.5 * IQR_value)] = Q3 + 1.5 *
IQR_value

df_outliers$Scores[df_outliers$Scores < (Q1 - 1.5 * IQR_value)] = Q1 - 1.5 *
IQR_value

print(df_outliers)
```

# Lab Sheet 04

## 1. One-Sample t-Test

- To determine if the mean of a single sample differs from a specified value

```
# Create vector to hold plant heights
my_data = c(14, 14, 16, 13, 12, 17, 15, 14, 15, 13, 15, 14)


# Perform one-sample t-test
t.test(my_data, mu = 15)
```

- mu - The hypothesized mean value
- Output includes the t-statistic, degrees of freedom, p-value, and confidence interval

---

## 2. Two-Sample t-Test

- To compare the means of two independent samples

```
# Create vectors for plant heights from two groups
group1 = c(8, 8, 9, 9, 9, 11, 12, 13, 13, 14, 15, 19)
group2 = c(11, 12, 13, 13, 14, 14, 14, 15, 16, 18, 18, 19)


# Perform two-sample t-test
t.test(group1, group2, var.equal = TRUE)
```

- var.equal - Specifies whether to assume equal variances
- Output includes comparison of means, confidence intervals, and p-value

---

**3. One-Sample z-Test**

- To test if the mean of a sample differs from a known population mean when the population standard deviation is known

```
# Load BSDA library

library(BSDA)


# IQ levels for 20 patients

data = c(88, 92, 94, 94, 96, 97, 97, 97, 99, 99, 105, 109, 109, 109, 110, 112, 112, 113, 114, 115)


# Perform one-sample z-test

z.test(data, mu = 100, sigma.x = 15)
```

- mu - Population mean
- sigma.x - Known standard deviation

---

**4. Two-Sample z-Test**

- To compare the means of two independent samples with known population standard deviations

```
# IQ levels for individuals in two cities

cityA = c(82, 84, 85, 89, 91, 91, 92, 94, 99, 99, 105, 109, 109, 109, 110, 112, 112, 113, 114, 114)

cityB = c(90, 91, 91, 91, 95, 95, 99, 99, 108, 109, 109, 114, 115, 116, 117, 117, 128, 129, 130, 133)
```

```
# Perform two-sample z-test
z.test(x = cityA, y = cityB, mu = 0, sigma.x = 15, sigma.y = 15)
```

---

**5. Variance Ratio Test (F-Test)**

- To compare variances of two independent samples

```
# Plant heights from two groups
group1 = c(5, 6, 6, 8, 10, 12, 12, 13, 14, 15, 15, 17, 18, 18, 19)
group2 = c(9, 9, 10, 12, 12, 13, 14, 16, 16, 19, 22, 24, 26, 29, 29)

# Perform variance ratio test
var.test(group1, group2)
```

---

**6. One-Proportion z-Test**

- To test if a population proportion differs from a specified value

```
# Proportion of COVID-positive patients
prop.test(x = 60, n = 100, p = 0.5, alternative = "greater")
```

- x - Number of successes
- n - Sample size
- p - Hypothesized proportion

---

**7. Two-Proportions z-Test**

- To compare proportions between two groups

```
# Proportions of COVID-positive patients in two groups
```

```
prop.test(x = c(60, 80), n = c(100, 200), alternative = "greater")
```

---

**8. Analysis of Variance (ANOVA)**

- To compare means across multiple groups

```
# Load iris dataset
data("iris")
colnames(iris) = c("SL", "SW", "PL", "PW", "Specs")
st_data = stack(iris[, 1:4])

# Perform ANOVA
ano_tab = aov(values ~ ind, data = st_data)
summary(ano_tab)
```

---

**9. Levene's Test**

- To test equality of variances

```
# Load car library
library(car)

# Perform Levene's test
leveneTest(values ~ ind, data = st_data)
```

---

**10. Post-Hoc Analysis - Fisher's LSD**

- To perform pairwise comparisons after ANOVA

```
# Create data frame
df = data.frame(technique = rep(c("tech1", "tech2", "tech3"), each = 10),
                score = c(72, 73, 73, 77, 82, 82, 83, 84, 85, 89,
                          81, 82, 83, 83, 83, 84, 87, 90, 92, 93,
                          77, 78, 79, 88, 89, 90, 91, 95, 95, 98))

# Fit one-way ANOVA
model = aov(score ~ technique, data = df)
summary(model)

# Fisher's LSD
library(agricolae)
LSD.test(model, "technique")
```

---

**11. Testing Normality**

- To check if data follows a normal distribution

```
# Generate random data from normal distribution
data = rnorm(100)
shapiro.test(data)
hist(data, col = "steelblue")

# Check normality for iris dataset
shapiro.test(iris$Sepal.Length)
```

```
hist(iris$Sepal.Length)
```

---

**12. Correlation Analysis**

- To measure the strength and direction of relationships between variables

```
# Data frame
df = data.frame(hours = c(1, 1, 3, 2, 4, 3, 5, 6),
                prac_exams = c(4, 3, 3, 2, 3, 2, 1, 4),
                score = c(69, 74, 74, 70, 89, 85, 99, 90))


# Pearson correlation
cor(df$hours, df$score)
cor(df)


# Correlation test
x = c(2, 3, 3, 5, 6, 9, 14, 15, 19, 21, 22, 23)
y = c(23, 24, 24, 23, 17, 28, 38, 34, 35, 39, 41, 43)


# Scatterplot
plot(x, y, pch = 16, col = "red")
cor.test(x, y)
```

# Lab Sheet 05

## Part 01

### Step 1: Load the Required Libraries

```
library(MASS)
```

The MASS package contains datasets like Boston that we'll use in this lab.

---

### Step 2: Load the Data

```
data("Boston")
```

The Boston dataset includes housing data with variables like crime rate, tax, and median home value (medv).

---

### Simple Linear Regression

Simple Linear Regression is used to understand the relationship between one independent variable (lstat) and the dependent variable (medv).

**Code**

```
fit1 = lm(medv ~ lstat, data = Boston)
```

```
summary(fit1)
```

**Explanation**

- lm() creates a linear model.
- medv ~ lstat means predicting medv using lstat.
- summary(fit1) displays key metrics like coefficients and p-values.

---

### Multiple Linear Regression

This extends simple linear regression by using multiple independent variables.

**Code Example 1**

```
fit2 = lm(medv ~ lstat + black, data = Boston)
summary(fit2)
```

**Code Example 2 (Using All Variables)**

```
fit3 = lm(medv ~ ., data = Boston)
summary(fit3)
```

**Excluding Specific Variables**

```
fit4 = lm(medv ~ . - indus, data = Boston)
summary(fit4)
```

**Explanation**

- medv ~ . uses all variables except the target (medv).
- - indus removes the indus variable.

---

**Splitting Data for Training and Testing**

**Code**

```
trainID = sample(1:nrow(Boston), round(0.8 * nrow(Boston)))
train = Boston[trainID, ]
test = Boston[-trainID, ]
```

**Explanation**

- sample() splits the data into 80% training and 20% testing datasets.

---

**Model Training and Predictions**

**Code**

```
fit = lm(medv ~ ., data = train)
summary(fit)


y_pred = predict(fit, test)
y_actual = test$medv
```

**Explanation**

- predict() estimates values for the test dataset.

---

**Model Evaluation**

Calculate errors to assess model performance.

**Code**

```
MSE = mean((y_actual - y_pred)^2)
RMSE = sqrt(MSE)
MSE
RMSE
```

**Explanation**

- **MSE** (Mean Squared Error): Average of squared errors.
- **RMSE** (Root Mean Squared Error): Square root of MSE.

---

**Handling Dummy Variables**

**Dataset**

```
library(pROC)
data("aSAH")
```

```
aSAH$outcome = factor(aSAH$outcome)

aSAH$gender = factor(aSAH$gender)

contrasts(aSAH$gender)
```

**Model**

```
fit = lm(ndka ~ s100b + outcome + gender, data = aSAH)

summary(fit)
```

**Explanation**

- Convert categorical variables to factors for regression.
- contrasts() shows how R handles dummy variables.

**Part 02**

**Assumption 1: Linearity**

The relationship between the independent and dependent variables should be linear.

**Code**

```
plot(Boston$lstat, Boston$medv, main = "Scatterplot of lstat vs medv",

   xlab = "lstat", ylab = "medv")

abline(lm(medv ~ lstat, data = Boston), col = "red")
```

**Explanation**

- A scatterplot shows the relationship between the predictor (lstat) and the response (medv).
- The red line is the fitted regression line. The points should follow a linear pattern.

---

**Assumption 2: Independence**

The residuals (errors) should be independent.

**Code**

```
library(lmtest)

dwtest(lm(medv ~ lstat, data = Boston))
```

**Explanation**

- The Durbin-Watson test checks for autocorrelation in the residuals.
- A test statistic close to 2 indicates no autocorrelation.

---

**Assumption 3: Homoscedasticity**

The residuals should have constant variance.

**Code**

```
fit = lm(medv ~ lstat, data = Boston)

plot(fitted(fit), residuals(fit), main = "Residuals vs Fitted Values",

   xlab = "Fitted Values", ylab = "Residuals")

abline(h = 0, col = "red")
```

**Explanation**

- A residuals vs. fitted values plot is used to check homoscedasticity.
- The residuals should be randomly scattered around zero.
- Patterns (e.g., funnel shapes) indicate heteroscedasticity.

---

**Assumption 4: Normality of Residuals**

The residuals should be normally distributed.

**Code**

```
qqnorm(residuals(fit), main = "Normal Q-Q Plot")
qqline(residuals(fit), col = "red")
```

**Explanation**

- A Q-Q plot compares the distribution of residuals to a normal distribution.
- The points should fall along the red reference line.

**Additional Test**

```
shapiro.test(residuals(fit))
```

- The Shapiro-Wilk test checks for normality. A p-value > 0.05 indicates normality.

---

**Assumption 5: No Multicollinearity**

Independent variables should not be highly correlated.

**Code**

```
library(car)
vif(lm(medv ~ ., data = Boston))
```

**Explanation**

- Variance Inflation Factor (VIF) checks multicollinearity.

VIF values > 5 indicate significant multicollinearity

# Lab Sheet 06

## Assumption 1: Linearity

The relationship between the independent and dependent variables should be linear.

### Code

```
plot(Boston$lstat, Boston$medv, main = "Scatterplot of lstat vs medv",
    xlab = "lstat", ylab = "medv")
abline(lm(medv ~ lstat, data = Boston), col = "red")
```

### Explanation

- A scatterplot shows the relationship between the predictor (lstat) and the response (medv).
- The red line is the fitted regression line. The points should follow a linear pattern.

## Assumption 2: Independence

The residuals (errors) should be independent.

### Code

```
library(lmtest)
dwtest(lm(medv ~ lstat, data = Boston))
```

### Explanation

- The Durbin-Watson test checks for autocorrelation in the residuals.
- A test statistic close to 2 indicates no autocorrelation.

## Assumption 3: Homoscedasticity

The residuals should have constant variance.

**Code**

```
fit = lm(medv ~ lstat, data = Boston)
plot(fitted(fit), residuals(fit), main = "Residuals vs Fitted Values",
    xlab = "Fitted Values", ylab = "Residuals")
abline(h = 0, col = "red")
```

**Explanation**

- A residuals vs. fitted values plot is used to check homoscedasticity.
- The residuals should be randomly scattered around zero.
- Patterns (e.g., funnel shapes) indicate heteroscedasticity.

**Assumption 4: Normality of Residuals**

The residuals should be normally distributed.

**Code**

```
qqnorm(residuals(fit), main = "Normal Q-Q Plot")
qqline(residuals(fit), col = "red")
```

**Explanation**

- A Q-Q plot compares the distribution of residuals to a normal distribution.
- The points should fall along the red reference line.

**Additional Test**

```
shapiro.test(residuals(fit))
```

- The Shapiro-Wilk test checks for normality. A p-value > 0.05 indicates normality.

**Assumption 5: No Multicollinearity**

Independent variables should not be highly correlated.

**Code**

```
library(car)

vif(lm(medv ~ ., data = Boston))
```

**Explanation**

- Variance Inflation Factor (VIF) checks multicollinearity.
- VIF values > 5 indicate significant multicollinearity.

# Lab Sheet 07

## Load the Data

First, we will load the 'Hitters' dataset from the ISLR library and handle any missing values.

```
library(ISLR)

data("Hitters")

sum(is.na(Hitters)) # Check for missing values


Hitters = na.omit(Hitters) # Remove rows with missing values

nrow(Hitters) # Number of rows after cleaning
```

---

## Best Subset Selection

We will use the leaps package to perform Best Subset Selection and evaluate the model performance using different metrics such as RSS, R-squared, Cp, BIC, and Adjusted R-squared.

```
library(leaps)

best = regsubsets(Salary~., data = Hitters, nvmax = 19) # Perform Best Subset Selection

results = summary(best)


# Model selection metrics

RSS = results$rss

r2 = results$rsq

Cp = results$cp
```

```
BIC = results$bic
Adj_r2 = results$adjr2
cbind(RSS, r2, Cp, BIC, Adj_r2) # Combine and display metrics

# Identify the best models
which.min(Cp) # Model with minimum Cp
which.min(BIC) # Model with minimum BIC
which.max(Adj_r2) # Model with maximum Adjusted R-squared

coef(best, 10) # Coefficients for the model with 10 predictors (based on Cp)
coef(best, 6)  # Coefficients for the model with 6 predictors (based on BIC)
coef(best, 11) # Coefficients for the model with 11 predictors (based on Adjusted R-squared)
```

---

**Forward Stepwise Selection**

Next, we will use the Forward Stepwise Selection method to select the best subset of predictors.

```
fwd = regsubsets(Salary~., data = Hitters, nvmax = 19, method = "forward")
results = summary(fwd)

# Model selection metrics
RSS = results$rss
r2 = results$rsq
Cp = results$cp
```

```
BIC = results$bic
Adj_r2 = results$adjr2
cbind(RSS, r2, Cp, BIC, Adj_r2)

# Identify the best models
which.min(Cp)
which.min(BIC)
which.max(Adj_r2)

coef(fwd, 10) # Coefficients for the model with 10 predictors (based on Cp)
coef(fwd, 6)  # Coefficients for the model with 6 predictors (based on BIC)
coef(fwd, 11) # Coefficients for the model with 11 predictors (based on Adjusted R-squared)
```

---

**Backward Stepwise Selection**

Finally, we will use the Backward Stepwise Selection method to identify the best subset of predictors.

```
bwd = regsubsets(Salary~., data = Hitters, nvmax = 19, method = "backward")
results = summary(bwd)
# Model selection metrics
RSS = results$rss
r2 = results$rsq
Cp = results$cp
BIC = results$bic
```

```
Adj_r2 = results$adjr2

cbind(RSS, r2, Cp, BIC, Adj_r2)

# Identify the best models

which.min(Cp)

which.min(BIC)

which.max(Adj_r2)


coef(bwd, 10) # Coefficients for the model with 10 predictors (based on Cp)

coef(bwd, 8)  # Coefficients for the model with 8 predictors (based on BIC)

coef(bwd, 11) # Coefficients for the model with 11 predictors (based on
Adjusted R-squared)
```

# Lab Sheet 08

## Load and Prepare the Data

We will use the built-in "mtcars" dataset for this lab. The dataset contains various measurements of car performance and design.

```
# Load the mtcars dataset

data("mtcars")

head(mtcars) # View the first few rows


# Standardize the data

mtcars_scaled = scale(mtcars)

head(mtcars_scaled) # View the scaled data
```

---

## Perform PCA

We will use the prcomp() function to perform PCA on the scaled dataset.

```
# Perform PCA

pca_result = prcomp(mtcars_scaled, center = TRUE, scale. = TRUE)


# Summary of PCA results

summary(pca_result)


# View the rotation matrix (principal components)

pca_result$rotation
```

```
# View the principal component scores
head(pca_result$x)
```

---

**Visualize PCA Results**

We will use the factoextra and ggplot2 packages to visualize the PCA results.

```
# Install and load the required libraries
library(factoextra)
library(ggplot2)

# Scree plot to visualize variance explained by each principal component
fviz_eig(pca_result)

# PCA biplot
fviz_pca_biplot(pca_result, label = "var", addEllipses = TRUE)

# PCA individual plot
fviz_pca_ind(pca_result, addEllipses = TRUE)

# PCA variable plot
fviz_pca_var(pca_result)
```

---

**Reconstruct Data Using Principal Components**

If needed, we can reconstruct the original data using a subset of principal components.

```
# Reconstruct the data using the first two principal components

reconstructed_data = pca_result$x[, 1:2] %*% t(pca_result$rotation[, 1:2])

# Add the mean back to the reconstructed data

reconstructed_data = scale(reconstructed_data, center = -colMeans(mtcars), scale = FALSE)

head(reconstructed_data)
```